

上海财经大学

信息管理与工程学院

随机梯度、Mini-batch 与 Adam

优化理论与算法 后续专题 大规模机器学习优化

王震

2026

课程导入：为什么需要随机梯度

前面学习的梯度下降默认每一步都能算出全梯度。但在机器学习中，目标函数常常来自很多样本的平均损失：

$$f(w) = \frac{1}{n} \sum_{i=1}^n f_i(w).$$

全梯度的瓶颈

每更新一次参数都要遍历全部 n 个样本。当 n 很大时，一步全梯度的代价可能已经很高。

自然想法

能不能不先计算所有样本的梯度，而是随机抽取一个或一小批样本，用它们估计当前下降方向，并立即更新参数？

本讲主线与目标

核心问题

当全梯度太贵时，如何用更便宜的随机方向高效、稳定地训练模型？

- ▶ 主线：从有限和目标出发，比较 GD、SGD、Mini-batch 三种更新；
- ▶ 目标一：理解随机抽样为什么能给出有用但有噪声的下降方向；
- ▶ 目标二：掌握 batch size、epoch、iteration 和学习率调度等训练语言；
- ▶ 目标三：了解 Momentum、RMSProp、Adam/AdamW 如何改进随机梯度训练。

补充动机：数据不断到来时怎么办

前面的主线是固定数据集：已经有一批样本，目标是最小化有限和平均损失。现实中还有另一类问题：数据不是一次性给完，而是持续到来。

- ▶ 推荐系统：不断出现新的用户请求、点击、购买和停留时间；
- ▶ 广告系统：每一次曝光、点击和转化都带来新的反馈；
- ▶ 金融或交易系统：新订单、新价格和新风险不断进入；
- ▶ 控制与仿真：每条环境轨迹都受到随机扰动影响。

关键区别

固定数据集的问题是“全梯度太贵”；在线数据的问题是“完整数据或真实分布根本无法一次看完”。因此不能等全部数据收集完再开始优化。

在线数据中的优化目标

在线场景下，我们关心的是模型在未来数据上的长期平均表现，而不是只在某个固定训练集上表现好。

$$\min_w F(w) = \mathbb{E}_\xi[\ell(w; \xi)].$$

- ▶ ξ : 一次随机出现的数据或环境结果，例如一次用户请求、一笔交易、一条轨迹；
- ▶ $\ell(w; \xi)$: 当前模型参数 w 在这次样本上的损失；
- ▶ $\mathbb{E}$: 对未来可能出现的样本取平均。

只作为补充视角

这个期望公式帮助我们描述在线/随机环境；本讲主线仍然先从有限和目标出发，学习 SGD 与 Mini-batch 的更新方式。

从新样本到随机梯度

如果每次只能看到新来的一个样本或一小批样本，最自然的做法是：先用当前看到的数据估计下降方向，再马上更新模型。

一个样本的方向

第 k 步拿到新样本 ξ_k 后，计算

$$g_k = \nabla \ell(w^k; \xi_k),$$

并把 g_k 当作当前平均下降方向的估计。

一小批样本的方向

如果一次拿到一小批样本，就平均它们的梯度，方向通常更稳定。这正好对应后面要讲的 Mini-batch。

接下来先回到固定数据集的有限和目标，把 GD、SGD 和 Mini-batch 三种更新式写清楚。

有限和目标：大规模学习的标准形式

很多经验风险最小化问题可以写成

$$f(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n f_i(\mathbf{w}),$$

其中 $f_i(\mathbf{w})$ 是第 i 个样本产生的损失。

例子：线性回归

$$f_i(\mathbf{w}) = \frac{1}{2}(\mathbf{x}_i^T \mathbf{w} - y_i)^2.$$

例子：分类模型

$$f_i(\mathbf{w}) = \ell(y_i, h_{\mathbf{w}}(\mathbf{x}_i)).$$

例如逻辑回归、神经网络的交叉熵损失。

全梯度需要累加所有样本：

$$\nabla f(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(\mathbf{w}).$$

GD 与 SGD

按“每次看多少样本”比较更新。公式较长时，纵向展开比三列并排更清楚。

GD: 使用全部样本

$$w^{k+1} = w^k - \alpha_k \left(\frac{1}{n} \sum_{i=1}^n \nabla f_i(w^k) \right)$$

方向准，但每一步都要遍历全部数据。

SGD: 使用一个样本

$$w^{k+1} = w^k - \alpha_k \nabla f_{i_k}(w^k)$$

每步便宜，但方向噪声大。

Mini-batch 更新

Mini-batch: 使用一小批样本

$$w^{k+1} = w^k - \alpha_k \left(\frac{1}{|B_k|} \sum_{i \in B_k} \nabla f_i(w^k) \right)$$

在计算效率和梯度稳定性之间折中；现代深度学习最常用。

符号提醒： α_k 是第 k 步的学习率，也就是本课程前面常说的步长。

从“每步准确”到“单位时间有效”

现在已经看过 GD、SGD 和 Mini-batch 的更新式。比较优化方法时，不只看“每一步方向准不准”，还要看“同样时间内能做多少次有效更新”。

方法	单步代价	训练直觉
GD	需要计算全部 n 个样本的梯度	一步方向准，但更新频率低
SGD	只计算 1 个样本的梯度	一步方向噪声大，但可以很快更新很多步
Mini-batch	计算一小批样本的平均梯度	借助并行计算，在速度和稳定性之间折中

小结

当全梯度的单步成本成为主要瓶颈时，我们可以接受“有噪声但便宜”的方向，用更多次快速更新换取更好的单位时间进展。

课堂案例（1）：背景——机器要学什么？

问题：给定一张手写数字图片，让计算机判断它写的是几。

- ▶ 人类：看笔画形状，几乎瞬间识别；
- ▶ 计算机：只能处理数字，必须把“图片”变成特征向量，再学一个函数做预测。

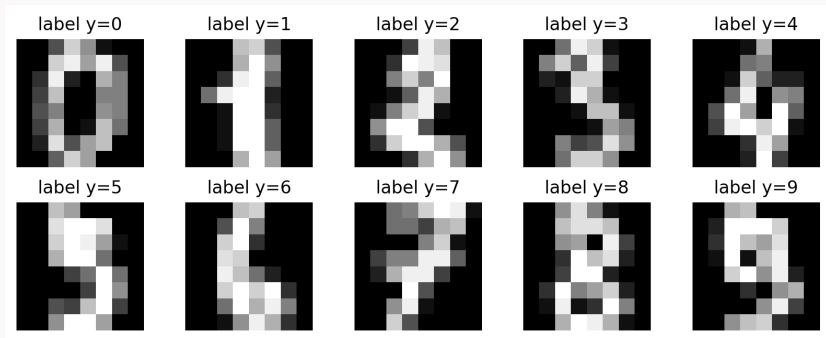
本课使用的数据集

`sklearn.datasets.load_digits`（常称 **Digits**）

- ▶ 共 $n \approx 1797$ 张图片，每张 8×8 灰度图（像素值 0–16）；
- ▶ 标签为 $0, 1, \dots, 9$ ，来自早年邮政信封手写邮编识别任务；
- ▶ 规模不大，但足够在课堂和 Notebook 里亲手跑通 GD / SGD / Mini-batch。

课堂案例（2）：数据长什么样？

下面每张图是一个训练样本；标签 y 就是“正确答案”（这张图写的是几）。



课堂案例（2）续：为什么需要很多样本？

先建立直觉

同一数字 7 每个人写法不同（笔画粗细、倾斜、缺口都不一样），模型不能“背下”某一张图。

必须从很多样本里学共同规律，这正是经验风险最小化：

$$\min_w \frac{1}{n} \sum_{i=1}^n f_i(w).$$

思考题：如果训练集只有 10 张图，全梯度 GD 和 SGD 的差别还明显吗？为什么？

课堂案例（2）续：为什么需要很多样本？

先建立直觉

同一数字 7 每个人写法不同（笔画粗细、倾斜、缺口都不一样），模型不能“背下”某一张图。

必须从很多样本里学共同规律，这正是经验风险最小化：

$$\min_w \frac{1}{n} \sum_{i=1}^n f_i(w).$$

思考题：如果训练集只有 10 张图，全梯度 GD 和 SGD 的差别还明显吗？为什么？

简答：差别会变小，因为全量梯度本身也不贵；但 SGD 仍会因只看少数样本而更抖。

课堂案例（3）：从图片到特征向量

神经网络可直接吃图片；本案例先用线性模型，因此要把 8×8 矩阵变成向量。



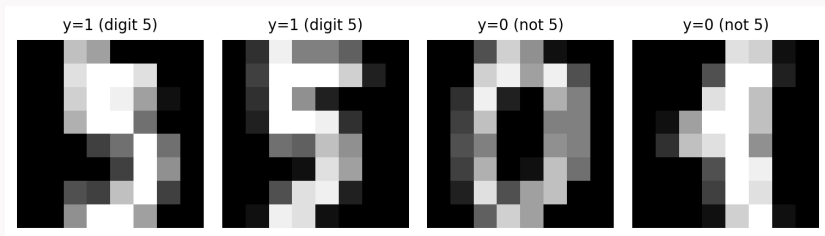
按行拼接： $x_i = (p_{i,1}, \dots, p_{i,64})^T \in \mathbb{R}^{64}$ 。Notebook 中还会做标准化（减均值、除标准差），避免某些像素维度数值过大主导梯度。

课堂案例（4）：标签怎么处理？

原始标签：第 i 张图对应 $y_i \in \{0, 1, \dots, 9\}$ （多分类）。

本实验先做二分类：判断“是不是数字 5”，构造

$$\tilde{y}_i = \begin{cases} 1, & y_i = 5, \\ 0, & y_i \neq 5. \end{cases}$$



课堂案例（4）续：多分类推广

认 0-9 十个类别时，常见做法是 **one-hot** 编码：

$$y_i \in \{0, \dots, 9\} \Rightarrow \text{向量}(0, \dots, 1, \dots, 0) \in \mathbb{R}^{10},$$

再配合 softmax 得到 10 类概率。优化目标仍是有限和：

$$f(w) = \frac{1}{n} \sum_{i=1}^n f_i(w).$$

说明：先讲二分类，公式更短，便于对照逻辑回归与交叉熵；多分类只是标签与输出层更复杂，随机梯度思想不变。

课堂案例（5）：预测函数与单样本损失

线性模型 + sigmoid: 对第 i 张图, 预测 “是 5” 的概率为

$$p_i(w) = \sigma(w^T x_i) = \frac{1}{1 + e^{-w^T x_i}}, \quad w \in \mathbb{R}^{64}.$$

若真实为 $\tilde{y}_i \in \{0, 1\}$, 用交叉熵衡量预测好坏:

$$f_i(w) = -\left[\tilde{y}_i \log p_i(w) + (1 - \tilde{y}_i) \log(1 - p_i(w))\right].$$

$\tilde{y}_i = 1$ 时希望 $p_i(w) \rightarrow 1$; $\tilde{y}_i = 0$ 时希望 $p_i(w) \rightarrow 0$ 。

课堂案例（5）续：经验风险与梯度

优化目标（全体训练图上的平均损失）：

$$\min_w f(w), \quad f(w) = \frac{1}{n} \sum_{i=1}^n f_i(w), \quad n \approx 1797.$$

全梯度（GD 每步用这个）：

$$\nabla f(w) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(w).$$

随机梯度（SGD 每步只抽一个 i_k ）：

$$g_k = \nabla f_{i_k}(w^k).$$

和本讲核心概念的对应

一张图 $\Leftrightarrow$ 一个 f_i ；一批图 $\Leftrightarrow$ 一个 Mini-batch 的平均梯度。

课堂案例（6）：三种更新在本任务上

方法	每步用多少张图	单步成本	实验中常见现象
GD	$n \approx 1797$	高	loss 曲线较平滑，但迭代少、耗时长
SGD	1	低	更新很快，曲线明显抖动
Mini-batch	32 等	中	折中：比 SGD 稳，比 GD 快

课堂案例：配套实验与思考题

配套 Notebook:

`notebooks/chapter8_stochastic_gradient_minibatch_experiments.ipynb`

- ▶ 实验 2: 对比 GD、SGD、Mini-batch 的 loss 曲线与耗时;
- ▶ 实验 3: 改变 `batch_size`, 观察每 epoch 的 iteration 与曲线抖动;
- ▶ 课堂案例补充: 在 Digits 上训练逻辑回归, 查看训练/测试 loss 与准确率。

跑完实验后思考:

- ▶ SGD 的 loss 为什么通常比 GD “更毛”? 这和单样本梯度噪声有什么关系?
- ▶ batch size 从 $1 \rightarrow 32 \rightarrow 256$: 曲线变稳了吗? 每个 epoch 的 iteration 变了吗?
- ▶ “每步梯度更准” 是否一定带来更快、更好的训练? 你更关注 iteration 数还是实际耗时?
- ▶ 优化算法除了 “方向准不准”, 还应关注哪些训练现象?

课堂案例：配套实验与思考题

配套 Notebook:

`notebooks/chapter8_stochastic_gradient_minibatch_experiments.ipynb`

- ▶ 实验 2: 对比 GD、SGD、Mini-batch 的 loss 曲线与耗时;
- ▶ 实验 3: 改变 `batch_size`, 观察每 epoch 的 iteration 与曲线抖动;
- ▶ 课堂案例补充: 在 Digits 上训练逻辑回归, 查看训练/测试 loss 与准确率。

跑完实验后思考:

- ▶ SGD 的 loss 为什么通常比 GD “更毛”? 这和单样本梯度噪声有什么关系?
- ▶ batch size 从 $1 \rightarrow 32 \rightarrow 256$: 曲线变稳了吗? 每个 epoch 的 iteration 变了吗?
- ▶ “每步梯度更准” 是否一定带来更快、更好的训练? 你更关注 iteration 数还是实际耗时?
- ▶ 优化算法除了 “方向准不准”, 还应关注哪些训练现象?

提示: 更 “毛” 通常对应噪声更大; batch 变大后曲线更稳但 iteration 更少, 还要结合实际耗时和验证集表现判断。

SGD 在做什么？

回忆有限和目标：

$$f(w) = \frac{1}{n} \sum_{i=1}^n f_i(w).$$

GD 每步用全梯度 $\nabla f(w^k)$ ；**SGD** 每步随机抽一个样本 i_k ，用

$$g_k = \nabla f_{i_k}(w^k)$$

更新

$$w^{k+1} = w^k - \alpha_k g_k.$$

关键问题： g_k 只来自一个样本，它凭什么还能让 $f(w)$ 整体下降？

随机梯度为什么可行 (1): 无偏性

若 i_k 在 $\{1, \dots, n\}$ 上均匀随机抽取, 则

$$\mathbb{E}_{i_k} [\nabla f_{i_k}(w)] = \sum_{i=1}^n \frac{1}{n} \nabla f_i(w) = \frac{1}{n} \sum_{i=1}^n \nabla f_i(w) = \nabla f(w).$$

结论

在任意当前参数 w 处, 随机梯度 g_k 是全梯度 $\nabla f(w)$ 的无偏估计: 平均而言, 它指向与 GD 相同的下降方向。

随机梯度为什么可行（2）：无偏就能下降吗？

无偏不等于每一步都准，但意味着：

- ▶ 随机误差不会“系统性”指向上升方向；
- ▶ 走很多步后，噪声分量会相互抵消，平均轨迹仍沿 $\nabla f(w)$ 方向前进；
- ▶ 因此 loss 可以总体下降（在适当条件和递减步长下，凸问题有 $\mathbb{E}[f(\bar{w}^k) - f^*] = \mathcal{O}(1/\sqrt{k})$ 等结果）。

直观类比

GD：每步都看完整张地图；SGD：每步只问一个人指路，单次可能偏，但长期平均仍朝目标走。

随机梯度为什么可行 (3): **Mini-batch** 同样无偏

Mini-batch 梯度

$$g_{B_k}(w) = \frac{1}{|B_k|} \sum_{i \in B_k} \nabla f_i(w)$$

若 B_k 中样本也是随机抽取, 则

$$\mathbb{E}[g_{B_k}(w)] = \nabla f(w).$$

与 SGD 的差别主要在方差: batch 越大, g_{B_k} 越接近全梯度, 曲线通常更稳。

$$\text{噪声大小} \approx \mathcal{O}\left(\frac{1}{\sqrt{|B_k|}}\right) \text{ (直觉量级)}.$$

思考题：随机方向为什么还能下降

想象有 n 位同学，每人根据一张训练图给出“下一步往哪走”的建议。

GD：问全班

方向最稳，但每步都要汇总全部意见，成本高。

SGD：随机问一人

每步很快；被问到的人若意见偏激，这一步就会偏，loss 曲线会抖。

思考题：随机问 1 人 vs 随机问 32 人，哪一种更容易抖动？哪一种每步更便宜？

思考题：随机方向为什么还能下降

想象有 n 位同学，每人根据一张训练图给出“下一步往哪走”的建议。

GD：问全班

方向最稳，但每步都要汇总全部意见，成本高。

SGD：随机问一人

每步很快；被问到的人若意见偏激，这一步就会偏，loss 曲线会抖。

思考题：随机问 1 人 vs 随机问 32 人，哪一种更容易抖动？哪一种每步更便宜？

简答：随机问 1 人更容易抖动、每步更便宜；问 32 人方向更稳，但单步成本更高。

噪声：把随机梯度拆成两部分

固定当前参数 w^k ，把 SGD 方向写成

$$g_k = \nabla f_{i_k}(w^k) = \underbrace{\nabla f(w^k)}_{\text{真正想要的平均方向}} + \underbrace{\varepsilon_k}_{\text{抽样带来的误差}},$$

其中

$$\varepsilon_k := \nabla f_{i_k}(w^k) - \nabla f(w^k).$$

无偏性即 $\mathbb{E}[\varepsilon_k] = 0$ ：误差长期平均后没有固定偏向。

噪声：均值为 0 究竟是什么意思？

- ▶ $\mathbb{E}[\varepsilon_k] = 0$ 不是 $\varepsilon_k = 0$ （单步误差通常不为 0）；
- ▶ 有的步 ε_k 与 $\nabla f(w^k)$ 同向（步子更猛），有的步反向（甚至暂时上升 loss）；
- ▶ 关键是：正负误差不会长期偏向一边，所以整体仍能下降。

batch size 从 1 增到 32、128： ε_k 的波动（方差）通常变小，loss 曲线更平滑，但每 epoch 的更新次数变少。

SGD 噪声的两面性

好处

- ▶ 便宜：每步只算一个或少量样本的梯度；
- ▶ 可扩展：数据可流式到来，不必每次都扫全库；
- ▶ 随机性：非凸问题中有时帮助跳出差的驻点或平坦区。

代价

- ▶ 抖动：单步方向方差大，loss 曲线不如 GD 平滑；
- ▶ 调参难：学习率、batch size 对稳定性更敏感；
- ▶ 不单调：可能出现某一步 loss 暂时上升。

思考题：Notebook 里 SGD 的 loss 更“毛”，对应的是 ε_k 大还是小？增大 batch 后 ε_k 如何变化？

SGD 噪声的两面性

好处

- ▶ 便宜：每步只算一个或少量样本的梯度；
- ▶ 可扩展：数据可流式到来，不必每次都扫全库；
- ▶ 随机性：非凸问题中有时帮助跳出差的驻点或平坦区。

代价

- ▶ 抖动：单步方向方差大，loss 曲线不如 GD 平滑；
- ▶ 调参难：学习率、batch size 对稳定性更敏感；
- ▶ 不单调：可能出现某一步 loss 暂时上升。

思考题：Notebook 里 SGD 的 loss 更“毛”，对应的是 ε_k 大还是小？增大 batch 后 ε_k 如何变化？

简答：更“毛”对应 ε_k 波动较大；增大 batch 后，平均会抵消一部分噪声， ε_k 的方差通常变小。

训练过程怎么数：Iteration、Batch、Epoch

随机优化里先分清三个计数（见 Notebook 实验 3）：

概念	含义
iteration	参数更新一次（每抽一个 batch 并更新，计 1 次）
batch B_k	第 k 次更新所用的样本集合
batch size $ B_k $	该批样本个数
epoch	训练集 n 个样本各被用到约一遍

一个 epoch 的 iteration 数约为 $\left\lceil \frac{n}{|B_k|} \right\rceil$ 。例如 $n = 1797$ 、 $|B_k| = 32$ 时，一个 epoch 约 57 次 iteration。

Batch size 如何影响训练？

回顾: $g_{B_k}(w) = \frac{1}{|B_k|} \sum_{i \in B_k} \nabla f_i(w)$, 且 $\mathbb{E}[g_{B_k}] = \nabla f$ 。

batch size	梯度噪声	每个 epoch 的更新次数
小 (如 1)	大, loss 曲线抖	多
中 (如 32)	中等, 常用	中等
大 (如 512)	小, 更平滑	少

思考题: batch 变大后, 单步梯度更准了, 但一个 epoch 内更新次数变少——训练一定更快吗?

Batch size 如何影响训练？

回顾: $g_{B_k}(w) = \frac{1}{|B_k|} \sum_{i \in B_k} \nabla f_i(w)$, 且 $\mathbb{E}[g_{B_k}] = \nabla f$ 。

batch size	梯度噪声	每个 epoch 的更新次数
小 (如 1)	大, loss 曲线抖	多
中 (如 32)	中等, 常用	中等
大 (如 512)	小, 更平滑	少

思考题: batch 变大后, 单步梯度更准了, 但一个 epoch 内更新次数变少——训练一定更快吗?

简答: 不一定。大 batch 可能提高硬件吞吐、降低噪声, 但更新次数少、显存占用高, 还可能需要重新调学习率。

SGD 的收敛性：定理陈述

定理 1.1: 凸光滑情形下的典型期望收敛率（课堂版）

$f(w) = \frac{1}{n} \sum_{i=1}^n f_i(w)$, 并满足有界初始距离等常规条件。若

1. f 为凸函数, 且 L -光滑 (梯度 Lipschitz);
2. 随机梯度方差有界: $\mathbb{E} \|\nabla f_{i_k}(w) - \nabla f(w)\|^2 \leq \sigma^2$;
3. 使用递减步长 $\alpha_k = \frac{c}{\sqrt{k}}$ ($c > 0$ 为常数)。

令 $\bar{w}^k = \frac{1}{k} \sum_{t=1}^k w^t$ 为前 k 步参数的平均。则

$$\mathbb{E}[f(\bar{w}^k) - f(w^*)] \leq \frac{C}{\sqrt{k}} = \mathcal{O}\left(\frac{1}{\sqrt{k}}\right).$$

常数 C 与初始距离、噪声方差等有关。强凸且配合 $\alpha_k \propto 1/k$ 时, 可得到更快的 $\mathcal{O}(1/k)$ 期望速率; 本讲先掌握“递减步长抑制噪声”的直觉。

证明思路 (1a): 单步下降不等式

目标: 说明在合适 α_k 下, 随机梯度虽然有噪声, 但平均意义上仍能带来下降。

$$f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \frac{L}{2} \|y - x\|^2.$$

把 x 换成 w^k , 并代入 SGD 更新

$$w^{k+1} = w^k - \alpha_k g_k,$$

得到

$$f(w^{k+1}) \leq f(w^k) - \alpha_k \langle \nabla f(w^k), g_k \rangle + \frac{L\alpha_k^2}{2} \|g_k\|^2.$$

读法

内积项表示“沿 g_k 走能下降多少”; 平方范数项来自光滑性, 是步子太大时要付出的二阶代价。

证明思路 (1b): 取条件期望

固定当前点 w^k 后, 随机性只来自抽到的样本, 因此可以先对 g_k 取条件期望。

$$\mathbb{E}[g_k \mid w^k] = \nabla f(w^k)$$

表示随机梯度平均起来等于真梯度, 所以

$$\mathbb{E}[\langle \nabla f(w^k), g_k \rangle \mid w^k] = \|\nabla f(w^k)\|^2.$$

于是

$$\mathbb{E}[f(w^{k+1}) \mid w^k] \leq f(w^k) - \alpha_k \|\nabla f(w^k)\|^2 + \frac{L\alpha_k^2}{2} \mathbb{E}[\|g_k\|^2 \mid w^k].$$

读法: 现在只剩一个麻烦项: 随机梯度的二阶矩 $\mathbb{E}[\|g_k\|^2 \mid w^k]$ 。

证明思路 (2a): 目标——从单步到总量控制

把随机梯度写成“真梯度 + 噪声”:

$$g_k = \nabla f(w^k) + \varepsilon_k, \quad \mathbb{E}[\varepsilon_k \mid w^k] = 0.$$

展开平方范数:

$$\|g_k\|^2 = \|\nabla f(w^k)\|^2 + 2\langle \nabla f(w^k), \varepsilon_k \rangle + \|\varepsilon_k\|^2.$$

取条件期望后, 交叉项为 0:

$$\mathbb{E}[\|g_k\|^2 \mid w^k] = \|\nabla f(w^k)\|^2 + \mathbb{E}[\|\varepsilon_k\|^2 \mid w^k] \leq \|\nabla f(w^k)\|^2 + \sigma^2.$$

课堂直觉: 随机梯度的二阶矩 = 真方向大小 + 噪声强度; SGD 分析就是用学习率控制噪声项累积。

证明思路 (2b): 从单步不等式到累加

把二阶矩上界代回上一页，噪声主要以 $\alpha_k^2 \sigma^2$ 的形式出现。若 α_k 不太大，可得到类似

$$\mathbb{E}[f(\mathbf{w}^{k+1})] \leq \mathbb{E}[f(\mathbf{w}^k)] - c_1 \alpha_k \mathbb{E} \|\nabla f(\mathbf{w}^k)\|^2 + c_2 \alpha_k^2 \sigma^2.$$

先移项：把真正想控制的梯度范数放到左边：

$$c_1 \alpha_k \mathbb{E} \|\nabla f(\mathbf{w}^k)\|^2 \leq \mathbb{E}[f(\mathbf{w}^k)] - \mathbb{E}[f(\mathbf{w}^{k+1})] + c_2 \alpha_k^2 \sigma^2.$$

再对 $k = 1, \dots, K$ 累加：

$$c_1 \sum_{k=1}^K \alpha_k \mathbb{E} \|\nabla f(\mathbf{w}^k)\|^2 \leq \sum_{k=1}^K \left(\mathbb{E}[f(\mathbf{w}^k)] - \mathbb{E}[f(\mathbf{w}^{k+1})] \right) + c_2 \sigma^2 \sum_{k=1}^K \alpha_k^2.$$

读法：左边是 K 步累计的“有效下降”；右边由函数值下降和噪声预算组成。

证明思路 (2c): 望远镜求和到最终结论 (课堂版)

假设回顾 (课堂版): f 为 L -光滑; 随机梯度无偏 $\mathbb{E}[g_k | w^k] = \nabla f(w^k)$; 噪声二阶矩有界 $\mathbb{E}\|g_k - \nabla f(w^k)\|^2 \leq \sigma^2$; 且步长足够小, 使上一页单步不等式成立。

上一页右边第一项会像望远镜一样首尾相消:

$$\begin{aligned} & \sum_{k=1}^K \left(\mathbb{E}[f(w^k)] - \mathbb{E}[f(w^{k+1})] \right) \\ &= \mathbb{E}[f(w^1)] - \mathbb{E}[f(w^{K+1})] \leq f(w^1) - f^*. \end{aligned}$$

把它代回上一页并除以 $\sum_{k=1}^K \alpha_k$, 得到“按学习率加权的平均梯度范数”界:

$$\frac{\sum_{k=1}^K \alpha_k \mathbb{E}\|\nabla f(w^k)\|^2}{\sum_{k=1}^K \alpha_k} \leq \frac{f(w^1) - f^*}{c_1 \sum_{k=1}^K \alpha_k} + \frac{c_2 \sigma^2 \sum_{k=1}^K \alpha_k^2}{c_1 \sum_{k=1}^K \alpha_k}.$$

证明思路 (2d): 最终结论怎么读 (典型速率)

再给一个常用的“最小梯度”版本: 因为 $\min_{1 \leq k \leq K} a_k \leq \frac{\sum_{k=1}^K \alpha_k a_k}{\sum_{k=1}^K \alpha_k}$ ($\alpha_k > 0$), 所以

$$\min_{1 \leq k \leq K} \mathbb{E} \|\nabla f(w^k)\|^2 \leq \frac{f(w^1) - f^*}{c_1 \sum_{k=1}^K \alpha_k} + \frac{c_2 \sigma^2 \sum_{k=1}^K \alpha_k^2}{c_1 \sum_{k=1}^K \alpha_k}.$$

- ▶ 两种读法: 加权平均 (更看重大学习率的步); 或随机抽点 R (按权重抽出某一步), 两者本质上对应同一个左边量。
- ▶ 项 1: $\frac{f(w^1) - f^*}{\sum \alpha_k}$, 累计步长越大, 初始差距摊得越薄;
- ▶ 项 2: $\frac{\sigma^2 \sum \alpha_k^2}{\sum \alpha_k}$, 步长平方累积越大, 噪声越难压住;
- ▶ 典型折中: 若取递减步长 (课件前面提到的 $\alpha_k = c/\sqrt{k}$), 两项会随 K 变小, 得到随迭代数改善的 $\mathcal{O}(1/\sqrt{K})$ 直觉 (噪声项常带一个 $\log K$ 的缓慢因子)。

学习率 α_k ：为什么 **SGD** 特别依赖它？

更新式 $w^{k+1} = w^k - \alpha_k g_k$ 中， α_k 同时影响两件事：

- 下降幅度：太小则进步慢，太大则可能越过最优点甚至发散；
- 噪声放大：项 $\frac{L\alpha_k^2}{2}\|g_k\|^2$ 表明， α_k 过大时随机噪声被放大，loss 更易抖动。

因此 SGD 常配合学习率调度：前期大步探索，后期小步稳定；这与上一页定理中 $\alpha_k \propto 1/\sqrt{k}$ 的精神一致（Notebook 实验 4）。

学习率调度：常见形式

设第 k 次 iteration 的学习率为 α_k ，常见写法如下。

策略	典型形式
固定	$\alpha_k \equiv \alpha_0$
反比衰减	$\alpha_k = \frac{\alpha_0}{\sqrt{k}}$ 或 $\frac{\alpha_0}{k}$
分段衰减	每过 T 个 epoch, $\alpha \leftarrow \eta\alpha$ (如 $\eta = 0.1$)
学习率预热 (warmup)	训练初期从较小学习率逐步升到目标 α_0 ，避免初始梯度不稳定导致发散
余弦衰减 (cosine decay)	按余弦曲线从较大值平滑降到较小值: $\alpha_k = \alpha_{\min} + \frac{1}{2}(\alpha_0 - \alpha_{\min})(1 + \cos(\pi k/K))$ ，后期更细致收敛

学习率调度：怎么选？

- ▶ 固定 α_0 ：实现简单；SGD 后期往往震荡，适合先观察 loss 曲线。
- ▶ $\alpha_k \propto 1/\sqrt{k}$ ：与凸 SGD 理论分析常见步长一致，理论友好。
- ▶ 分段衰减：深度学习中最常用之一（如每 30 epoch 乘 0.1）。
- ▶ 学习率预热 + 余弦衰减：大模型预训练常见；先稳住开局，再平滑减小步长。

思考题：若 batch size 增大，梯度噪声变小，学习率通常应增大还是减小？为什么？

学习率调度：怎么选？

- ▶ 固定 α_0 ：实现简单；SGD 后期往往震荡，适合先观察 loss 曲线。
- ▶ $\alpha_k \propto 1/\sqrt{k}$ ：与凸 SGD 理论分析常见步长一致，理论友好。
- ▶ 分段衰减：深度学习中最常用之一（如每 30 epoch 乘 0.1）。
- ▶ 学习率预热 + 余弦衰减：大模型预训练常见；先稳住开局，再平滑减小步长。

思考题：若 batch size 增大，梯度噪声变小，学习率通常应增大还是减小？为什么？

简答：实践中常可适当增大学习率，因为平均梯度更稳；但不能无限增大，过大仍会越过谷底或发散，通常要配合预热。

案例（1）：推荐系统在预测什么？

场景：信息流、短视频、电商首页——需要预测用户是否会点击/购买某商品。

每条日志是一条样本，例如

$$\text{样本 } i: (u_i, v_i, y_i),$$

u_i 为用户， v_i 为物品， $y_i \in \{0, 1\}$ 表示是否点击。

embedding（嵌入向量/嵌入表示）：把离散对象（用户、商品、词、类别 ID）映射成可训练的低维连续向量，模型借此表示相似性或语义关系。

用用户/物品 embedding 与模型 h_w 得到点击概率 $\hat{p}_i$ ，单样本损失常为交叉熵：

$$f_i(w) = \ell_{\text{CE}}(y_i, \hat{p}_i).$$

案例（2）：推荐系统的有限和与 Mini-batch

全体日志构成有限和

$$f(w) = \frac{1}{n} \sum_{i=1}^n f_i(w), \quad n \text{ 可达百万、千万, 且每天增长.}$$

一次 **Mini-batch** 更新:

1. 从当日（或滑动窗口）日志中随机抽取 batch B_k ;
2. 计算 $g_{B_k}(w^k) = \frac{1}{|B_k|} \sum_{i \in B_k} \nabla f_i(w^k)$;
3. 更新用户/物品 embedding: $w^{k+1} = w^k - \alpha_k g_{B_k}(w^k)$ 。

一个 epoch 常指“扫过一遍当前可用训练日志”；在线系统中 epoch 会不断刷新。

案例（3）：为什么推荐几乎必用 SGD？

不适合全量 GD

- ▶ 数据持续新增，全量梯度每步太贵；
- ▶ 兴趣分布漂移，需要频繁小步更新；
- ▶ 特征多为高维稀疏 embedding。

Mini-batch 的价值

- ▶ 每几分钟即可用新日志更新一轮；
- ▶ batch 大小受 GPU 显存约束（如 512–4096）；
- ▶ 常配合学习率调度与 Adam/AdamW。

思考题：若把 batch 从 512 提到 8192，训练日志里的 iteration/epoch 数如何变化？对“模型多久更新一次”有何影响？

案例（3）：为什么推荐几乎必用 SGD？

不适合全量 GD

- ▶ 数据持续新增，全量梯度每步太贵；
- ▶ 兴趣分布漂移，需要频繁小步更新；
- ▶ 特征多为高维稀疏 embedding。

Mini-batch 的价值

- ▶ 每几分钟即可用新日志更新一轮；
- ▶ batch 大小受 GPU 显存约束（如 512–4096）；
- ▶ 常配合学习率调度与 Adam/AdamW。

思考题：若把 batch 从 512 提到 8192，训练日志里的 iteration/epoch 数如何变化？对“模型多久更新一次”有何影响？

简答：每个 epoch 的 iteration 数约按比例减少；若每次更新要等更大的 batch 凑齐，模型响应新数据的频率可能下降。

SGD 的广泛应用：一张地图

同一套思想 $f(w) = \frac{1}{n} \sum_i f_i(w) + \text{随机/小批量梯度}$ ，出现在很多看似不同的系统里：

领域	数据形态	为何几乎必用 SGD/Mini-batch
推荐 / 广告	海量点击日志、持续增长	全量 GD 太慢，需频繁在线更新
大语言模型	万亿 token，无法一次装入内存	只能按 token/序列 batch 训练
图像 / 语音	百万级样本、GPU 并行	Mini-batch 兼顾稳定与硬件吞吐
风控 / 欺诈	流式交易、概念漂移	样本源源不断，算法必须“边来边学”
强化学习	与环境交互生成轨迹	梯度本身由随机 rollout 估计

案例：大语言模型预训练（数据读不完）

任务：用下一个词预测（或类似目标）训练大模型，语料规模可达万亿 token。
若用全量 GD，一步更新要先“看完”当前全部语料——这在现实中不可行：

$$n \text{ 极大} \Rightarrow \text{一次全梯度的代价} \propto n.$$

实际做法：每个 iteration 只取一个 Mini-batch 的文本片段（如几千 token），

$$w^{k+1} = w^k - \alpha_k g_{B_k}(w^k),$$

多卡并行时 batch 再按 GPU 拆分。

反直觉：模型“没见过全部互联网”，却仍能学到可用语言规律——靠的就是对海量样本的长期随机平均。

案例：在线广告与实时竞价

场景：搜索/信息流广告要在几十毫秒内决定展示哪条广告，并估计点击率 CTR。

样本是“曝光--点击”记录，目标仍是最小化平均 $\log \text{loss}$ ；但数据 **70E24** 小时涌入，旧数据快速过期。

- ▶ 全量 GD：等所有日志汇总再更新 $\Rightarrow$ 模型跟不上热点事件；
- ▶ Mini-batch SGD：每几分钟用最新一批曝光日志更新 $\Rightarrow$ CTR 模型及时适应。

反直觉：为了更准，反而不能每次用“全部历史数据”算梯度，而要刻意只用一小批最新样本。

案例：图像分类与 GPU 训练

任务：ResNet、ViT 等在 ImageNet 等数据集上训练（ $n \approx 10^6$ 量级）。

这里用 Mini-batch 不只为“省算力”，还为硬件：

- ▶ 单张图放进 GPU 显存容易，整数据集放不下；
- ▶ 一批图像矩阵运算可并行，吞吐远高于逐张更新（batch=1 的 SGD）；
- ▶ batch=256/512 常是“算法稳定性”与“GPU 利用率”折中的结果。

反直觉：batch 太大（如 8192）单步梯度更准，但每个 epoch 更新次数少，且可能泛化变差——“更准的梯度”不等于“更好的模型”。

案例：深度学习中的噪声与泛化

深度网络损失曲面非凸，SGD 的随机噪声不再只是“麻烦”，有时反而是正则化：

- ▶ 小 batch 的梯度噪声更大，参数不易陷入很窄的尖锐极小值；
- ▶ 平坦极小值往往泛化更好（对测试分布更稳）；
- ▶ 因此实践中常见“先用较大 batch 预训练，再调小 batch 或加噪声微调”。

反直觉：在 GD 里我们怕噪声；在深度学习的 SGD 里，适量噪声有时帮助测试集表现——与凸优化理论中的故事并不矛盾，因为问题类型不同。

案例：流式风控与强化学习（补充）

欺诈 / 风控

交易、支付行为源源不断到来，标签滞后、分布漂移。

用滑动窗口内的 Mini-batch 更新模型：

$$f(w) = \frac{1}{n} \sum_{i \in \text{窗口}} f_i(w),$$

窗口随时间滚动， n 与样本集都在变——天然就是随机优化场景。

强化学习

智能体与环境交互得到轨迹，回报对参数的梯度本身由随机采样轨迹估计。

可写成期望形式 $\min_w \mathbb{E}_\tau[\ell(w; \tau)]$ ，与“期望型随机优化”动机一致。

思考题：以上案例中，哪些是全量 GD 在算力上不可行？哪些是在时效上不可行？

案例：流式风控与强化学习（补充）

欺诈 / 风控

交易、支付行为源源不断到来，标签滞后、分布漂移。

用滑动窗口内的 Mini-batch 更新模型：

$$f(w) = \frac{1}{n} \sum_{i \in \text{窗口}} f_i(w),$$

窗口随时间滚动， n 与样本集都在变——天然就是随机优化场景。

强化学习

智能体与环境交互得到轨迹，回报对参数的梯度本身由随机采样轨迹估计。

可写成期望形式 $\min_w \mathbb{E}_\tau[\ell(w; \tau)]$ ，与“期望型随机优化”动机一致。

思考题：以上案例中，哪些是全量 GD 在算力上不可行？哪些是在时效上不可行？

简答：LLM、图像等主要受算力/内存限制；在线广告、推荐、风控还强烈受时效限制，因为数据和分布会持续变化。

拓展内容总览：SGD 之后还要解决什么

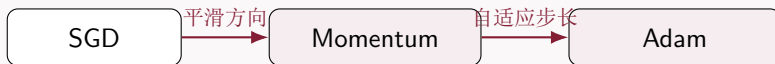
前面已经说明：SGD / Mini-batch 用低成本随机方向替代全梯度。接下来要进一步回答：方向有噪声、尺度不一致、训练很长时，优化器如何改进 **SGD**？

问题	代表方法	额外记忆量	核心价值
方向抖动、zig-zag	Momentum / Nesterov	历史梯度方向	平滑方向，沿稳定趋势前进
不同坐标尺度差异大	AdaGrad	历史平方梯度和	给每个坐标不同有效步长
长时间训练中步长过快衰减	RMSProp	近期平方梯度平均	只估计近期尺度，避免步长越来越小
既要方向平滑，又要坐标自适应	Adam / AdamW	一阶矩 + 二阶矩	工程中常用，启动快，适合大模型和稀疏参数

为什么还需要 Momentum 和 Adam

SGD 好用，但真实训练中常遇到三个问题：

- ▶ 梯度噪声大，方向抖动；
- ▶ 病态问题中 zig-zag 明显；
- ▶ 不同坐标梯度尺度差异大，一个全局学习率难以兼顾所有方向。



学习主线

Momentum 改变的是“如何使用过去的方向”；AdaGrad / RMSProp / Adam 改变的是“每个坐标走多大步”。

从“平均噪声”到“平滑方向”

SGD 的每一步都可能被当前 batch 的噪声影响。动量的想法是：不要完全相信当前这一批样本，而是把最近几步的梯度方向综合起来。

步数	1	2	3	4
随机梯度方向	右上	右下	右上	右下
共同趋势	都在向右，但上下方向来回变			

动量做了什么

动量会保留“持续向右”的共同趋势，同时削弱“上、下反复变化”的噪声部分。这就是它能减少 zig-zag 的直观原因。

Momentum: 公式怎么读

Momentum（动量）借用物理中“惯性/动量”的直觉：保留过去方向。动量 SGD 在参数 w 之外维护一个速度变量 v ：

$$v^{k+1} = \beta v^k + g_k,$$

$$w^{k+1} = w^k - \alpha_k v^{k+1}.$$

其中 $0 \leq \beta < 1$ ， g_k 是随机梯度或 Mini-batch 梯度；这里 v 是梯度方向的累积，真正的更新方向是 $-v$ 。

βv^k

过去方向的记忆。若 β 大，算法更相信历史趋势。

g_k

当前 batch 给出的新方向。它会修正历史速度。

价值：连续下降方向被累积，来回震荡方向被抵消，因此训练轨迹更平滑。

动量的指数加权解释

由递推式

$$v^{k+1} = \beta v^k + g_k$$

展开可得

$$v^{k+1} = g_k + \beta g_{k-1} + \beta^2 g_{k-2} + \cdots$$

因此 v^{k+1} 是历史梯度的指数加权累积，越近的梯度权重越大。若写成 $m^{k+1} = \beta m^k + (1 - \beta)g_k$ ，则是指数移动平均；两种写法只差一个可并入学习率的尺度。

β 小

更看重当前梯度，反应快但平滑弱。

β 大

历史记忆更强，更平滑但可能反应慢或冲过头。

案例：小批量训练中的 zig-zag

在随机优化中，我们真正关心的是经验风险

$$f(w) = \frac{1}{n} \sum_{i=1}^n f_i(w).$$

全梯度 $\nabla f(w)$ 是所有样本的平均方向，但 Mini-batch 梯度

$$g_{B_k}(w) = \frac{1}{|B_k|} \sum_{i \in B_k} \nabla f_i(w)$$

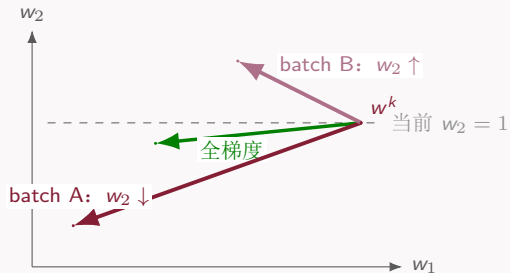
会随抽到的样本而改变。

为什么会 zig-zag

若不同 batch 的梯度在某坐标上符号经常相反，参数会在该方向来回摆动；另一方向若平均仍下降，轨迹就呈“向前走但左右晃”的 zig-zag（来自 Mini-batch 噪声，而非确定性二次函数）。

数值图示：不同 **batch** 的方向

设 $w^k = (1.0, 1.0)$ ，全梯度 $\nabla f(w^k) \approx (2.0, 0.2)$ ；两个 Mini-batch 给出 $g_A = (2.8, 1.0)$ 、 $g_B = (1.2, -0.6)$ 。取 $\alpha = 0.2$ 。



方向	g_2	$\Delta w_2 = -\alpha g_2$
batch A	1.0	-0.20
全梯度	0.2	-0.04
batch B	-0.6	+0.12

看图要点

两批样本在 w_2 方向一上一下；但 w_1 都向左，Mini-batch 方向围绕绿色的全梯度方向抖动。

数值图示：课堂理解

课堂理解

batch A 让 w_2 下降，batch B 反而让 w_2 上升。连续训练时，这种方向反复变化就会形成 zig-zag。

Momentum 的作用

把多次梯度做平滑平均，稳定的下降方向会被保留，来回摆动的噪声方向会被部分抵消。

Nesterov 动量：先看前方再修正

Nesterov 动量的思想是：先根据动量看一眼前方，再计算梯度。

$$\begin{aligned}\tilde{w}^k &= w^k - \alpha_k \beta v^k, & g_k^{\text{look}} &= \nabla f(\tilde{w}^k), \\ v^{k+1} &= \beta v^k + g_k^{\text{look}}, & w^{k+1} &= w^k - \alpha_k v^{k+1}.\end{aligned}$$

直观理解

普通动量像“先看当前坡度再冲”；Nesterov 像“先按惯性预判一下会到哪里，再看那里的坡度”。

课堂定位：Momentum 强调稳定训练方向；Nesterov 在此基础上增加“提前纠偏”的思想。不同框架可能把 v 定义为更新方向而非梯度累积，因此正负号会有等价写法。

自适应步长：共同模板

Momentum 主要处理“方向抖动”。另一个常见困难是：不同坐标的梯度尺度差异很大。

$$w_j^{k+1} := w_j^k - \alpha_k \frac{q_{k,j}}{s_{k,j} + \epsilon}, \quad j = 1, \dots, p.$$

$q_{k,j}$ 表示当前或平滑后的第 j 个梯度分量， $s_{k,j}$ 表示该坐标的梯度尺度估计；这是一条坐标公式，不是“向量整体相除”。

AdaGrad

用全部历史平方梯度估计尺度。

RMSProp

只用近期平方梯度估计尺度。

Adam

方向也平滑，尺度也自适应。

二维例子：为什么要按坐标缩放

设当前梯度 $g = (4, 0.4)$ ，基础学习率 $\alpha = 0.1$ 。若历史或近期平方梯度给出

$$r = (16, 0.16), \quad \sqrt{r} = (4, 0.4),$$

这里 $\sqrt{r}$ 是逐坐标开方，因此两个坐标的梯度尺度相差 10 倍。

方法	更新量	更新直觉
普通 SGD	$\Delta w = -0.1(4, 0.4) = (-0.40, -0.04)$	同一个学习率：第 1 坐标走得猛，第 2 坐标走得小。
自适应缩放	$\Delta w \approx -0.1 g / (\sqrt{r} + \epsilon) \approx (-0.10, -0.10)$	大尺度坐标被压小，小尺度或稀疏坐标仍能有效更新。

读法

自适应方法相当于使用坐标级有效学习率 $\alpha_j^{\text{eff}} = \alpha_k / (\sqrt{r_j} + \epsilon)$ 。这里第 1 坐标约为 0.025，第 2 坐标约为 0.25。

AdaGrad: 按坐标缩放步长

AdaGrad 的英文全称是 **Adaptive Gradient Algorithm**, 核心思想是: 每个坐标都有自己的有效学习率。

AdaGrad 记录每个坐标历史梯度平方和:

$$r_k = r_{k-1} + g_k \odot g_k.$$

第 j 个坐标更新为

$$w_j^{k+1} = w_j^k - \alpha_k \frac{g_{k,j}}{\sqrt{r_{k,j}} + \epsilon}, \quad j = 1, \dots, p.$$

- ▶ g_k : 当前随机梯度或 Mini-batch 梯度;
- ▶ $\odot$: 逐坐标相乘, 例如 $(a \odot b)_j = a_j b_j$;
- ▶ r_k : 每个坐标历史梯度平方的累积量;
- ▶ 向量式 $w^{k+1} = w^k - \alpha_k g_k / (\sqrt{r_k} + \epsilon)$ 只是简写, $\sqrt{\cdot}$ 、加 ϵ 和除法都按坐标计算;
- ▶ ϵ : 很小的正数, 防止除以 0。

AdaGrad 的坐标级直觉

若第 j 个坐标的历史梯度经常很大，则 $r_{k,j}$ 很大，有效步长

$$\frac{\alpha_k}{\sqrt{r_{k,j}} + \epsilon}$$

会变小。

直觉

某个坐标过去梯度经常很大，就减小该坐标步长；很少被更新的坐标保持相对较大的步长。

价值与局限

AdaGrad 对稀疏特征很有用；但 r_k 单调增大，训练很久后有效步长可能过小。

案例：文本模型中的稀疏特征

在文本分类或搜索排序中，特征可能是词语、短语或类别 ID。很多特征非常稀疏：

“通货膨胀”出现频率高， “近端梯度法”出现频率低。

频繁特征

历史梯度平方和大，AdaGrad 会自动减小步长，避免更新过猛。

稀有特征

历史梯度平方和小，仍保留较大学习率，更容易被学习到。

RMSProp: 只看近期梯度尺度

RMSProp 的英文全称通常写作 **Root Mean Square Propagation**。它改进 AdaGrad 的关键是：不要永久累加所有历史梯度平方，而是对近期梯度平方做指数平均。

RMSProp 使用

$$r_k = \rho r_{k-1} + (1 - \rho)(g_k \odot g_k), \quad 0 < \rho < 1.$$

第 j 个坐标更新为

$$w_j^{k+1} = w_j^k - \alpha_k \frac{g_{k,j}}{\sqrt{r_{k,j}} + \epsilon}, \quad j = 1, \dots, p.$$

- ▶ r_k : 近期梯度平方的指数移动平均;
- ▶ ρ : 遗忘系数, 常见取值如 0.9;
- ▶ $1 - \rho$: 当前梯度平方进入平均的权重;
- ▶ $\sqrt{r_{k,j}}$: 第 j 个坐标近期梯度的均方根尺度; 平方根、加 ϵ 和除法均为逐坐标运算。

RMSProp 和 AdaGrad 的差别

AdaGrad

$$r_k = r_{k-1} + g_k \odot g_k.$$

一直累加历史平方梯度, r_k 只增不减,
后期有效步长可能越来越小。

RMSProp

$$r_k = \rho r_{k-1} + (1 - \rho)(g_k \odot g_k).$$

逐渐遗忘很久以前的梯度, 更关注最近一段时间的尺度。

与 AdaGrad 的差别

RMSProp 可以看作“带遗忘机制的 AdaGrad”, 因此更适合长时间训练的神经网络。

Adam: 动量 + 自适应步长

Adam 的全称是 **Adaptive Moment Estimation** (自适应矩估计)。它可以看成把 Momentum 和 RMSProp 合在一起: 一条线负责“方向”, 一条线负责“尺度”。

一阶矩: 平滑方向

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k,$$

二阶矩: 估计尺度

$$v_k = \beta_2 v_{k-1} + (1 - \beta_2)(g_k \odot g_k).$$

偏差修正:

$$\hat{m}_k = \frac{m_k}{1 - \beta_1^k}, \quad \hat{v}_k = \frac{v_k}{1 - \beta_2^k}.$$

第 j 个坐标更新:

$$w_j^{k+1} = w_j^k - \alpha_k \frac{\hat{m}_{k,j}}{\sqrt{\hat{v}_{k,j}} + \epsilon}, \quad j = 1, \dots, p.$$

向量式写作 $\hat{m}_k/(\sqrt{\hat{v}_k} + \epsilon)$ 时, 平方根、加 ϵ 和除法也都逐坐标计算。

Adam: 公式怎么读

一阶矩 m_k

类似 Momentum, 减少随机梯度方向抖动。

二阶矩 v_k

类似 RMSProp, 估计每个坐标近期梯度尺度。

Adam 的更新可以口语化理解为:

平滑后的方向 / 每个坐标的近期梯度尺度.

常见默认值

$$\beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \epsilon = 10^{-8}.$$

Adam 中的偏差修正为什么出现

Adam 通常从 $m_0 = 0$, $v_0 = 0$ 开始。如果直接使用 m_k, v_k , 训练初期它们会被 0 拉小。

量	含义
m_k	刚开始偏小, 因为历史平均里混入了初始 0
$\hat{m}_k = m_k / (1 - \beta_1^k)$	把初期偏小的平均值校正回来
$\hat{v}_k = v_k / (1 - \beta_2^k)$	对平方梯度尺度做同样校正

课堂口径

偏差修正不是改变 Adam 的基本思想, 而是让训练刚开始的动量和尺度估计更可靠。

Adam 的一个坐标级小例子

忽略偏差修正，假设某一步中某个坐标的平滑梯度和平方梯度尺度为

$$m_{k,j} = 0.20, \quad v_{k,j} = 0.04.$$

若 $\alpha_k = 0.001$, ϵ 很小，则该坐标更新量近似为

$$\Delta w_j = -\alpha_k \frac{m_{k,j}}{\sqrt{v_{k,j}} + \epsilon} \approx -0.001 \cdot \frac{0.20}{0.20} = -0.001.$$

看懂这个例子

$m_{k,j}$ 给出方向和大小； $\sqrt{v_{k,j}}$ 表示这个坐标近期梯度尺度。若某坐标梯度尺度很大，Adam 会自动把这个坐标的有效步长压小。

案例：为什么 Adam 适合 embedding 参数

推荐系统和语言模型中常有大量 **embedding**（嵌入向量）参数：它们把用户、商品、词等离散 ID 变成可训练的低维连续向量。

- ▶ 热门商品或常见词：经常出现在 batch 中，梯度更新频繁；
- ▶ 长尾商品或低频词：很少出现，梯度更新稀疏；
- ▶ 不同 embedding 维度的梯度尺度可能差异很大。

为什么自适应方法有用

Adam 会为每个坐标估计近期梯度尺度，从而让频繁、大梯度坐标更稳，让稀疏坐标仍有机会被有效更新。

Adam 的价值：为什么工程中常用

- ▶ 对不同坐标尺度比较鲁棒；
- ▶ 对稀疏梯度、深层网络、embedding 参数通常较友好；
- ▶ 初期 loss 下降快，容易得到可用模型；
- ▶ 在 PyTorch、TensorFlow 等框架中默认支持，工程使用方便。

典型应用

大语言模型预训练、图像分类、推荐系统、广告点击率预测、强化学习中的神经网络训练等。

Adam 不是万能的：理解边界

Adam 在深度学习中非常常用，但需要注意：

- ▶ 对学习率仍然敏感；
- ▶ 训练 loss 快不一定代表泛化最好；
- ▶ 在某些任务上 SGD + momentum 的最终泛化表现可能更好；
- ▶ 权重衰减需要小心处理，实践中常用 AdamW；
- ▶ 理论收敛分析比 GD、SGD 更复杂。

实践建议

Adam 常适合快速得到可用模型；若追求最终泛化，可尝试学习率衰减、AdamW，或后期切换到 SGD + momentum。

AdamW: 深度学习中的常见变体

AdamW 常解释为 **Adam with decoupled weight decay**, 其中 W 对应 weight decay。它的思想是把权重衰减从 Adam 的自适应梯度更新中解耦出来。先记

$$u_k = \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon},$$

其中平方根、加 ϵ 和除法仍按坐标分别计算。AdamW 更新可写为

$$w^{k+1} := (1 - \alpha_k \lambda) w^k - \alpha_k u_k.$$

Adam

自适应梯度更新, 适合快速训练。

AdamW

更清晰地加入权重衰减, 是 Transformer、视觉模型中常见选择。

提醒: AdamW 不是把 λw^k 加进 g_k 后再经过 Adam 分母; 那是 Adam + L2 正则的耦合写法。

优化器选择：课堂决策表

场景	常见选择	选择理由
小规模、光滑、要求高精度	Newton、BFGS、L-BFGS	可利用曲率或近似曲率
大规模凸问题或线性模型	SGD、Mini-batch、AdaGrad	单步便宜，适合样本很多
深度网络初期训练	Adam、AdamW、RMSProp	启动快，对尺度较鲁棒
追求最终泛化	SGD + momentum	常配合学习率衰减精细训练
稀疏特征、embedding	AdaGrad、Adam、AdamW	坐标级步长更友好

选择原则

先看数据规模和梯度成本，再看噪声、坐标尺度、泛化需求和工程约束。

常见误区

误区	正确理解
Adam 一定比 SGD 好	Adam 初期常快，但最终泛化未必总优
batch size 越大越好	大 batch 梯度准，但每步贵，也可能需要更复杂学习率策略
loss 抖动说明算法失败	SGD 抖动是正常现象，需要结合趋势、epoch 和验证集判断
默认学习率总能用	优化器默认参数常能启动训练，但好结果仍需调参

从形式看算法：一页串起来

方法	更新形式	主要改进
SGD	$w^{k+1} = w^k - \alpha_k g_k$	用随机方向降低单步成本
Momentum	$v^{k+1} = \beta v^k + g_k$	平滑方向，减少 zig-zag
AdaGrad / RMSProp	$\Delta w_j = -\alpha_k g_{k,j} / (\sqrt{r_{k,j}} + \epsilon)$	每个坐标自适应步长
Adam	$\Delta w_j = -\alpha_k \hat{m}_{k,j} / (\sqrt{\hat{v}_{k,j}} + \epsilon)$	方向平滑 + 坐标自适应

学生需要记住的主线

先理解 SGD 为什么便宜但有噪声，再理解 Momentum 处理方向噪声，自适应方法处理坐标尺度，Adam 则把两类思想结合起来。

本讲总结

- ▶ 大规模机器学习目标常是有限和 $f(w) = \frac{1}{n} \sum_i f_i(w)$ 。
- ▶ GD 每步使用所有样本，方向准但代价高；SGD 和 Mini-batch 用噪声方向换取低成本更新。
- ▶ batch size、epoch、iteration 和学习率调度是理解训练过程的基本语言。
- ▶ Momentum / Nesterov 用历史方向平滑随机梯度，减少 zig-zag，并在稳定趋势上加速。
- ▶ AdaGrad、RMSProp 按坐标调整有效步长，适合梯度尺度差异大或稀疏特征场景。
- ▶ Adam / AdamW 同时使用一阶矩和二阶矩，是深度学习中常见的快速训练选择，但仍要结合泛化需求调参。

课后思考

为什么很多深度学习训练先用 Adam 快速下降，而一些任务后期又会尝试 SGD + momentum？

本讲总结

- ▶ 大规模机器学习目标常是有限和 $f(w) = \frac{1}{n} \sum_i f_i(w)$ 。
- ▶ GD 每步使用所有样本，方向准但代价高；SGD 和 Mini-batch 用噪声方向换取低成本更新。
- ▶ batch size、epoch、iteration 和学习率调度是理解训练过程的基本语言。
- ▶ Momentum / Nesterov 用历史方向平滑随机梯度，减少 zig-zag，并在稳定趋势上加速。
- ▶ AdaGrad、RMSProp 按坐标调整有效步长，适合梯度尺度差异大或稀疏特征场景。
- ▶ Adam / AdamW 同时使用一阶矩和二阶矩，是深度学习中常见的快速训练选择，但仍要结合泛化需求调参。

课后思考

为什么很多深度学习训练先用 Adam 快速下降，而一些任务后期又会尝试 SGD + momentum？

提示：Adam 前期对尺度差异鲁棒、启动快；SGD + momentum 后期步长更统一，有些任务上泛化更稳。